

Simulate

模拟

本页面将简要介绍模拟算法。

简介

模拟就是用计算机来模拟题目中要求的操作。

模拟题目通常具有码量大、操作多、思路繁复的特点。由于它码量大，经常会出现难以查错的情况，如果在考试中写错是相当浪费时间的。

技巧

写模拟题时，遵循以下的建议有可能会提升做题速度：

- 在动手写代码之前，在草纸上尽可能地写好要实现的流程。
- 在代码中，尽量把每个部分模块化，写成函数、结构体或类。
- 对于一些可能重复用到的概念，可以统一转化，方便处理：如，某题给你 "YY-MM-DD 时：分" 把它抽取到一个函数，处理成秒，会减少概念混淆。
- 调试时分块调试。模块化的好处就是可以方便的单独调某一部分。
- 写代码的时候一定要思路清晰，不要想到什么写什么，要按照落在纸上的步骤写。

实际上，上述步骤在解决其它类型的题目时也是很有帮助的。

例题详解

▼ [Climbing Worm](#)

一只长度不计的蠕虫位于 英寸深的井的底部。它每次向上爬 英寸，但是必须休息一次才能再次向上爬。在休息的时候，它滑落了 英寸。之后它将重复向上爬和休息的过程。蠕虫爬出井口需要至少爬多少次？如果蠕虫爬完后刚好到达井的顶部，我们也设作蠕虫已经爬出井口。

- 解题思路
- 参考代码

习题

- [「NOIP2014」生活大爆炸版石头剪刀布 - Universal Online Judge](#)
- [「OpenJudge 3750」魔兽世界](#)
- [「SDOI2010」猪国杀 - LibreOJ](#)

本页面最近更新：2024/3/27 06:46:12, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [NachtgeistW](#), [ChungZH](#), [Enter-tainer](#), [H-J-Granger](#), [i-yyi](#), [lr1d](#), [kenlig](#), [ksyx](#), [LeiJinpeng](#), [leoleoasd](#), [ouuan](#), [shawllewyw](#), [shuzhouliu](#), [Tiphereth-A](#), [WException](#), [zryi2003](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用